



**DETECTION AND CLASSIFICATION
OF COVID-19 AND PNEUMONIA
FROM CHEST X-RAY IMAGES USING
CONVOLUTIONAL NEURAL
NETWORKS**



SUBMITTED BY: TANUDIP GHOSH

ABSTRACT

This project presents a deep learning-based system for automatic classification of chest X-ray images into three categories: **COVID-19, Pneumonia, and Normal**. Convolutional Neural Networks (CNNs) are employed to extract image features and perform disease classification. Image preprocessing and data augmentation techniques are used to improve model generalization.

Experimental results demonstrate that CNN models can effectively distinguish between different respiratory conditions, assisting healthcare professionals in rapid diagnosis and screening.

CONTENTS

<u>1. INTRODUCTION</u>	3
<u>2. PROBLEM STATEMENT</u>	4
<u>3. OBJECTIVES</u>	4
<u>4. DATASET DESCRIPTION</u>	5
<u>5. DATA DISTRIBUTION</u>	6
<u>6. METHODOLOGY</u>	7
<u>7. SAMPLE CHEST X-RAY IMAGES FOR CLASSIFICATION</u>	9
<u>8. MODEL TRAINING & EVALUATION</u>	10
<u>9. MODEL PREDICTIONS ON TEST DATA</u>	21
<u>10. CONCLUSION</u>	23
<u>11.FUTURE SCOPE</u>	24

INTRODUCTION

Artificial Intelligence has revolutionized medical image analysis by enabling automated disease detection using deep learning techniques. Chest X-ray imaging is widely used for diagnosing respiratory diseases because it is cost-effective, non-invasive, and readily available.

COVID-19 and Pneumonia are serious respiratory illnesses that affect lung function and can lead to severe health complications if not diagnosed early. Traditional diagnosis relies heavily on radiologists, which may result in delays when large numbers of patients require screening.

Deep learning models, particularly Convolutional Neural Networks (CNNs), have shown remarkable success in image classification tasks. CNNs automatically learn important visual features from medical images, eliminating the need for manual feature extraction.

This project focuses on developing a CNN-based system capable of classifying chest X-ray images into three categories: COVID-19, Pneumonia, and Normal. The system aims to provide accurate and efficient disease detection that can support healthcare professionals during diagnosis.

PROBLEM STATEMENT

Accurate differentiation between COVID-19, Pneumonia, and Normal chest X-ray images is a challenging task. Manual diagnosis requires experienced radiologists and may become difficult when patient volume increases significantly. There is a need for an automated and reliable system that can assist healthcare professionals by rapidly classifying chest X-ray images and identifying respiratory diseases.

OBJECTIVES

1. To develop a deep learning model using Convolutional Neural Networks.
2. To classify chest X-ray images into COVID-19, Pneumonia, and Normal categories.
3. To perform image preprocessing and augmentation for improving model performance.
4. To evaluate the effectiveness of CNN-based classification.
5. To analyze model performance using standard evaluation metrics.
6. To assist healthcare professionals through automated disease detection.

DATASET DESCRIPTION

The dataset consists of chest X-ray images belonging to three classes:

- COVID-19
- Pneumonia
- Normal

The dataset is divided into training and testing subsets.



I have taken data for this statistical analysis. The link is given below:

<https://www.kaggle.com/datasets/pranavraikokte/covid19-image-dataset>

DATA DISTRIBUTION:

Training Data Distribution:

- Normal: 70 images
- Covid: 111 images
- Viral Pneumonia: 70 images

Testing Data Distribution:

- Normal: 20 images
- Covid: 26 images
- Viral Pneumonia: 20 images

Sample Image Dimensions:

The images in the dataset exhibit varying dimensions.

This variability suggests that image preprocessing (e.g., resizing) is necessary before feeding them into a convolutional neural network.

METHODOLOGY

The proposed system follows a deep learning approach for the classification of chest X-ray images into three categories: COVID-19, Viral Pneumonia, and Normal.

1. Dataset Collection

A chest X-ray dataset containing COVID-19, Viral Pneumonia, and Normal images was obtained from Kaggle. The dataset was divided into training and testing sets for model development and evaluation.

2. Image Preprocessing

All images were resized to 224×224 pixels and pixel values were normalized from 0-255 to 0-1 using rescaling. This improves training stability and model performance.

3. Data Augmentation

To reduce overfitting and improve generalization, data augmentation techniques such as random rotation, width shift, height shift, and zooming were applied to the training images.

4. CNN Model Development

A Convolutional Neural Network (CNN) was developed consisting of:

- Two Convolutional Layers (32 and 64 filters)
- Two Max-Pooling Layers
- Flatten Layer

- Two Dense Layers with 128 neurons each
- Output Layer with Softmax activation for three-class classification

The CNN automatically extracts important features from chest X-ray images and performs disease classification.

5. Model Training

The model was trained using the training dataset with the Adam optimizer and categorical cross-entropy loss function. Training and validation accuracy were monitored throughout the learning process.

6. Overfitting Prevention

Data augmentation, Early Stopping, and Model Checkpointing were used to prevent overfitting and improve model generalization. The best-performing model was automatically saved during training.

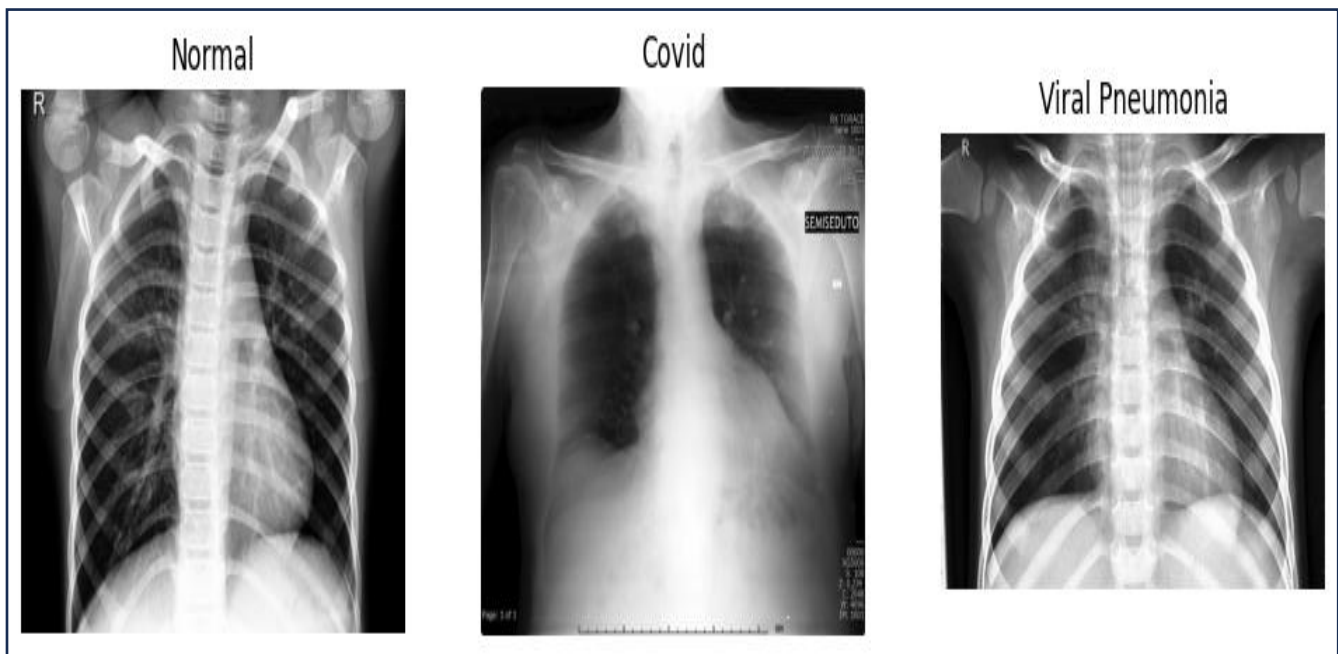
7. Performance Evaluation

The trained model was evaluated on unseen test data using accuracy and loss metrics. The final predictions were analyzed to assess the model's effectiveness in distinguishing between COVID-19, Viral Pneumonia, and Normal chest X-ray images.

SAMPLE CHEST X-RAY IMAGES FOR CLASSIFICATION

Below are representative samples of chest X-ray images from each of the three diagnostic categories: 'Normal', 'Covid', and 'Viral Pneumonia'. These images constitute the primary input data for our convolutional neural network (CNN) model.

Visual inspection of these samples can sometimes reveal distinct characteristics, such as different opacities or infiltrates, which the model is trained to identify for classification. However, a deep learning approach is necessary to capture the subtle and complex patterns that differentiate these conditions accurately.



MODEL TRAINING & EVALUATION

Rescaling: Image pixel values were rescaled from the range 0–255 to 0–1 by dividing each pixel value by 255. This normalization helps the model train more efficiently, improves numerical stability, and can lead to faster convergence and better performance.

```
generatoy=ImageDataGenerator(rescale=1/255)
```

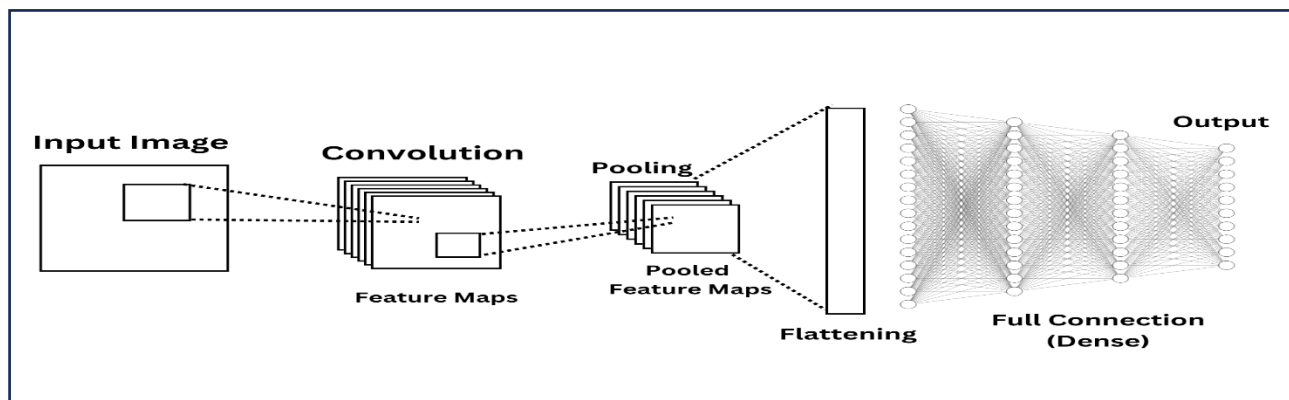
Model Architecture:

Our classification model is built using a Convolutional Neural Network (CNN), a type of deep learning model particularly well-suited for image analysis tasks.

CNNs are a special kind of neural network for processing data that has a known grid-like topology like images.

CNN layers:

1. Convolutional Layer
2. Pooling Layer
3. Fully Connected Layer



Our architecture consists of the following layers:

```
cnn_model=Sequential()

cnn_model.add(Conv2D(filters=32,kernel_size=3,padding='same',activation='relu',input_shape=[224,224,3]))
cnn_model.add(MaxPooling2D(pool_size=(2,2)))

cnn_model.add(Conv2D(filters=64,kernel_size=3,padding='same',activation='relu'))
cnn_model.add(MaxPooling2D(pool_size=(2,2)))

cnn_model.add(Flatten())

cnn_model.add(Dense(128,activation='relu'))
cnn_model.add(Dense(128,activation='relu'))

cnn_model.add(Dense(3,activation=softmax))
```

```
cnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
flatten (Flatten)	(None, 200704)	0
dense (Dense)	(None, 128)	25,690,240
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 3)	387

Total params: 25,726,531 (98.14 MB)

Trainable params: 25,726,531 (98.14 MB)

Non-trainable params: 0 (0.00 B)

The Convolutional Neural Network (CNN) was developed to automatically classify chest X-ray images into three categories. CNN was selected because it is one of the most effective deep learning techniques for image analysis and medical image classification. Unlike traditional machine learning methods, CNN

automatically extracts important features from images without requiring manual feature engineering.

The model accepts input images of size $224 \times 224 \times 3$. The architecture consists of convolutional layers, pooling layers, fully connected layers, and an output layer.

First Convolutional Layer

The first convolutional layer contains **32 filters** of size 3×3 with the ReLU activation function and same padding.

```
cnn_model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=[224, 224, 3]))
```

This layer scans the chest X-ray image and extracts low-level features such as edges, lines, textures, and intensity variations. These features form the foundation for identifying lung structures and abnormalities.

Padding: Padding is used to preserve important edge information during convolution.

Stride: Stride controls the step size of the filter as it moves across the image. A stride of 1 ensures detailed feature extraction.

Here, **Same padding** is used with default stride=1 to preserve the spatial dimensions of the input image during convolution. It prevents the loss of important edge information and allows deeper networks to extract features without rapidly reducing image size. This helps the CNN learn more effectively from chest X-ray images.

First Max-Pooling Layer

```
cnn_model.add(MaxPooling2D(pool_size=(2,2)))
```

The pooling layer reduces the image dimensions from 224×224 to 112×112 while retaining the most significant information. This reduces computational complexity and helps prevent overfitting.

Second Convolutional Layer

```
cnn_model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
```

The second convolutional layer contains **64 filters**, allowing the network to learn more complex and disease-specific patterns. At this stage, the model can detect features associated with pneumonia such as lung opacities, infiltrates, and abnormal textures.

Second Max-Pooling Layer

```
cnn_model.add(MaxPooling2D(pool_size=(2,2)))
```

This layer further reduces the feature map size from 112×112 to 56×56 , decreasing computational requirements while preserving critical information learned by the convolutional layers.

Flatten Layer

```
cnn_model.add(Flatten())
```

After feature extraction, the multi-dimensional feature maps are converted into a one-dimensional vector of 200,704 features. This transformation allows the extracted image features to be processed by fully connected neural network layers.

Fully Connected Layers

```
cnn_model.add(Dense(128,activation='relu'))  
cnn_model.add(Dense(128,activation='relu'))
```

Two dense layers with 128 neurons each are used to learn complex relationships among the extracted features. These layers combine the learned image characteristics and improve the model's ability to distinguish between different chest conditions.

The ReLU activation function is used because it introduces non-linearity, improves learning efficiency, and reduces the vanishing gradient problem.

Output Layer

```
cnn_model.add(Dense(3,activation=softmax))
```

The final layer contains 3 neurons corresponding to the three target classes. The Softmax activation function converts the output into probability values, and the class with the highest probability is selected as the final prediction.

The CNN architecture contains approximately **25.7 million trainable parameters**. The convolutional layers perform automatic feature extraction from chest X-ray images, while the dense layers perform classification based on these extracted features. The model is capable of learning both low-level visual patterns and high-level disease-specific characteristics, making it suitable for pneumonia detection and chest X-ray image classification.

Model Training

This model was then applied and trained on our designated training dataset.

cnn_model.compile(optimizer='adam',loss='categorical_crossentropy',metrics=['accuracy'])				
x=cnn_model.fit(train_data,epochs=10,validation_data=test_data)				
Epoch 1/10				
8/8	44s	5s/step	accuracy: 0.4661 - loss: 3.0427 - val_accuracy: 0.5152 - val_loss: 1.0147	
Epoch 2/10				
8/8	31s	4s/step	accuracy: 0.8048 - loss: 0.7072 - val_accuracy: 0.6970 - val_loss: 0.6604	
Epoch 3/10				
8/8	29s	4s/step	accuracy: 0.8964 - loss: 0.2881 - val_accuracy: 0.8636 - val_loss: 0.3825	
Epoch 4/10				
8/8	29s	4s/step	accuracy: 0.9602 - loss: 0.1204 - val_accuracy: 0.8485 - val_loss: 0.3972	
Epoch 5/10				
8/8	29s	3s/step	accuracy: 0.9522 - loss: 0.0912 - val_accuracy: 0.9545 - val_loss: 0.2038	
Epoch 6/10				
8/8	30s	3s/step	accuracy: 0.9960 - loss: 0.0255 - val_accuracy: 0.8939 - val_loss: 0.2774	
Epoch 7/10				
8/8	30s	4s/step	accuracy: 1.0000 - loss: 0.0089 - val_accuracy: 0.9545 - val_loss: 0.2530	
Epoch 8/10				
8/8	29s	4s/step	accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 0.8939 - val_loss: 0.2723	
Epoch 9/10				
8/8	29s	4s/step	accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 0.9091 - val_loss: 0.2193	
Epoch 10/10				
8/8	29s	4s/step	accuracy: 1.0000 - loss: 6.7696e-04 - val_accuracy: 0.9545 - val_loss: 0.2546	

The CNN model achieved 100% training accuracy after 10 epochs, indicating successful learning of the training data. However, the lower validation accuracy (up to 95.45%) and higher validation loss suggest mild **overfitting**, meaning the model performs better on training data than on unseen test data.

Overfitting may occur due to the large number of trainable parameters (25.7 million) relative to the dataset size. Techniques such as **data augmentation**, **dropout**, **batch normalization**, **regularization**, and **early stopping** can be employed to improve generalization and reduce overfitting.

Data Augmentation:

To reduce overfitting and improve the generalization capability of the proposed model, **data augmentation** techniques were applied during the training process. Data augmentation artificially increases the diversity of the training dataset by generating modified versions of existing images, enabling the model to learn more robust and invariant features without requiring additional data collection.

```
genertoy=ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=10,  
    width_shift_range=0.1,  
    height_shift_range=0.1,  
    horizontal_flip=False,  
    zoom_range=0.1,  
    fill_mode='nearest'  
)
```

Applied Augmentation Techniques

- **Rescaling (rescale=1./255)**
Normalizes pixel values from the range [0, 255] to [0, 1], which helps improve training stability and convergence.
- **Random Rotation (rotation_range=10)**
Rotates images randomly within ± 10 degrees, making the model less sensitive to slight orientation changes.
- **Width Shift (width_shift_range=0.1)**
Randomly shifts images horizontally by up to 10% of the image width, helping the model learn positional invariance.

- **Height Shift (height_shift_range=0.1)**
Randomly shifts images vertically by up to 10% of the image height, improving robustness to object displacement.
- **Random Zoom (zoom_range=0.1)**
Zooms images in or out by up to 10%, enabling the model to recognize objects at different scales.
- **Fill Mode (fill_mode='nearest')**
Fills newly created pixels after transformations using the nearest pixel values, preventing empty regions in augmented images.
- **Horizontal Flip (horizontal_flip=False)**
Horizontal flipping was intentionally disabled because flipping could alter important spatial characteristics of the images and potentially introduce incorrect training samples.

Impact on Model Performance

The use of data augmentation significantly reduced overfitting by exposing the model to a wider variety of image transformations during training. As a result, the model learned more generalized features rather than memorizing the training samples. This improved its performance on unseen validation and test data, leading to better robustness and reliability in real-world applications.

Early Stopping and Model Checkpointing

To further enhance the performance of the augmented modified model and prevent overfitting, **Early Stopping** and **Model Checkpointing** techniques were incorporated during training.

Early Stopping

Early Stopping is a regularization technique that monitors the model's performance on the validation dataset and automatically terminates training when further learning does not result in meaningful improvement. In this work, validation accuracy (val_accuracy) was used as the monitoring metric. Training was stopped when the validation accuracy failed to improve for five consecutive epochs, while the best model weights were restored automatically.

This approach offers the following advantages:

- Prevents the model from overfitting the training data.
- Reduces unnecessary training time and computational cost.
- Ensures that the model parameters corresponding to the best validation performance are retained.
- Improves the model's generalization ability on unseen data.

Model Checkpointing

Model Checkpointing was employed to save the model whenever an improvement in validation accuracy was observed. As a result, the best-performing version of the model was preserved during training and used for final evaluation.

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Define callbacks for CNN model
early_stopping_cnn = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    mode='max',
    verbose=1,
    restore_best_weights=True,
    baseline=0.95 # Stop if val_accuracy does not improve beyond 0.95
)

model_checkpoint_cnn = ModelCheckpoint(
    'best_cnn_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

x = cnn_model.fit(
    train_data,
    epochs=50, # Set a higher number of epochs, EarlyStopping will stop it automatically
    validation_data=test_data,
    callbacks=[early_stopping_cnn, model_checkpoint_cnn]
)

```

```

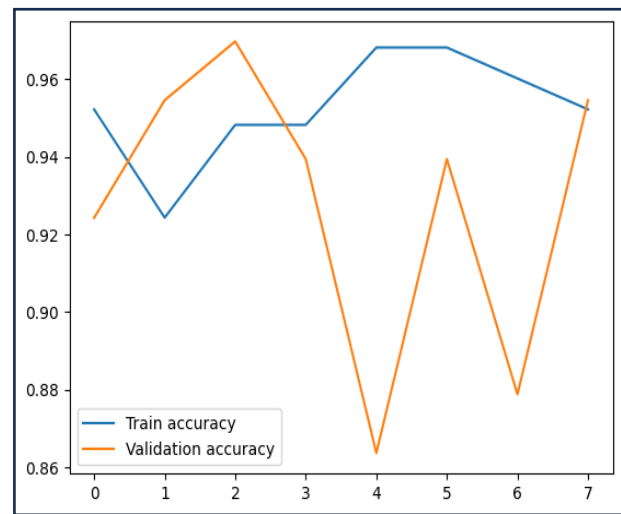
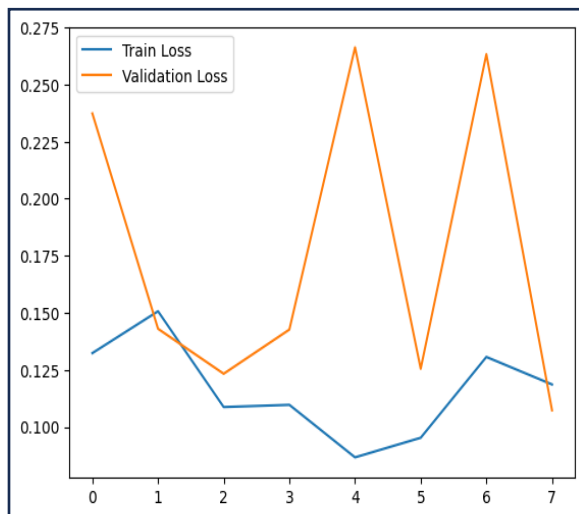
... Epoch 1/50
8/8 ----- 0s 2s/step - accuracy: 0.9561 - loss: 0.1328
Epoch 1: val_accuracy improved from None to 0.92424, saving model to best_cnn_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
Epoch 1: finished saving model to best_cnn_model.h5
8/8 ----- 21s 3s/step - accuracy: 0.9522 - loss: 0.1323 - val_accuracy: 0.9242 - val_loss: 0.2373
Epoch 2/50
8/8 ----- 0s 2s/step - accuracy: 0.9243 - loss: 0.1469
Epoch 2: val_accuracy improved from 0.92424 to 0.95455, saving model to best_cnn_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
Epoch 2: finished saving model to best_cnn_model.h5
8/8 ----- 20s 2s/step - accuracy: 0.9243 - loss: 0.1507 - val_accuracy: 0.9545 - val_loss: 0.1430
Epoch 3/50
8/8 ----- 0s 2s/step - accuracy: 0.9425 - loss: 0.1268
Epoch 3: val_accuracy improved from 0.95455 to 0.96970, saving model to best_cnn_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
Epoch 3: finished saving model to best_cnn_model.h5
8/8 ----- 21s 3s/step - accuracy: 0.9482 - loss: 0.1087 - val_accuracy: 0.9697 - val_loss: 0.1233
Epoch 4/50
8/8 ----- 0s 2s/step - accuracy: 0.9527 - loss: 0.0931
Epoch 4: val_accuracy did not improve from 0.96970
8/8 ----- 19s 2s/step - accuracy: 0.9482 - loss: 0.1097 - val_accuracy: 0.9394 - val_loss: 0.1426
Epoch 5/50
8/8 ----- 0s 2s/step - accuracy: 0.9590 - loss: 0.0864
Epoch 5: val_accuracy did not improve from 0.96970
8/8 ----- 19s 2s/step - accuracy: 0.9681 - loss: 0.0867 - val_accuracy: 0.8636 - val_loss: 0.2662
Epoch 6/50
8/8 ----- 0s 2s/step - accuracy: 0.9588 - loss: 0.1037
Epoch 6: val_accuracy did not improve from 0.96970
8/8 ----- 18s 2s/step - accuracy: 0.9681 - loss: 0.0952 - val_accuracy: 0.9394 - val_loss: 0.1255
Epoch 7/50
8/8 ----- 0s 2s/step - accuracy: 0.9651 - loss: 0.1032
Epoch 7: val_accuracy did not improve from 0.96970
8/8 ----- 20s 2s/step - accuracy: 0.9602 - loss: 0.1307 - val_accuracy: 0.8788 - val_loss: 0.2633
Epoch 8/50
8/8 ----- 0s 2s/step - accuracy: 0.9471 - loss: 0.1362
Epoch 8: val_accuracy did not improve from 0.96970
8/8 ----- 18s 2s/step - accuracy: 0.9522 - loss: 0.1186 - val_accuracy: 0.9545 - val_loss: 0.1073
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.

cnn_model.evaluate(test_data)
3/3 ----- 1s 241ms/step - accuracy: 0.9697 - loss: 0.1233
[0.12329641729593277, 0.969696985961914]

```

Performance Analysis

The augmented modified model achieved a training accuracy of 95.22% with a training loss of 0.1267. During training, the validation accuracy improved steadily and reached a maximum of 96.97%. The comparable training and validation accuracies indicate that the model generalized well to unseen data and did not suffer from overfitting. The use of data augmentation and Early Stopping contributed to improved robustness and stable model performance.



MODEL PREDICTIONS ON TEST DATA

In this section, we will evaluate the trained CNN model's performance on unseen test data. The objective is to demonstrate the model's capability in classifying chest X-ray images into 'Normal', 'Covid', or 'Viral Pneumonia' categories. We will visualize individual predictions to assess the model's accuracy.

```
num_samples_per_class = 3
all_images_to_plot = []
all_actual_labels = []
all_predicted_labels = []

for category in image_class:
    category_path = os.path.join(df_test, category)
    images_in_folder = os.listdir(category_path)

    # Take a few images for prediction
    sample_images = images_in_folder[:num_samples_per_class]

    for img_name in sample_images:
        test_image_path = os.path.join(category_path, img_name)

        testing_img = image.load_img(test_image_path, target_size = (224, 224))
        test_img_array = image.img_to_array(testing_img) / 255
        test_img_input = test_img_array.reshape(1, test_img_array.shape[0],
                                                test_img_array.shape[1], test_img_array.shape[2])

        predicted_class_idx = np.argmax(cnn_model.predict(test_img_input, verbose=0))
        predicted_label = class_map[predicted_class_idx]

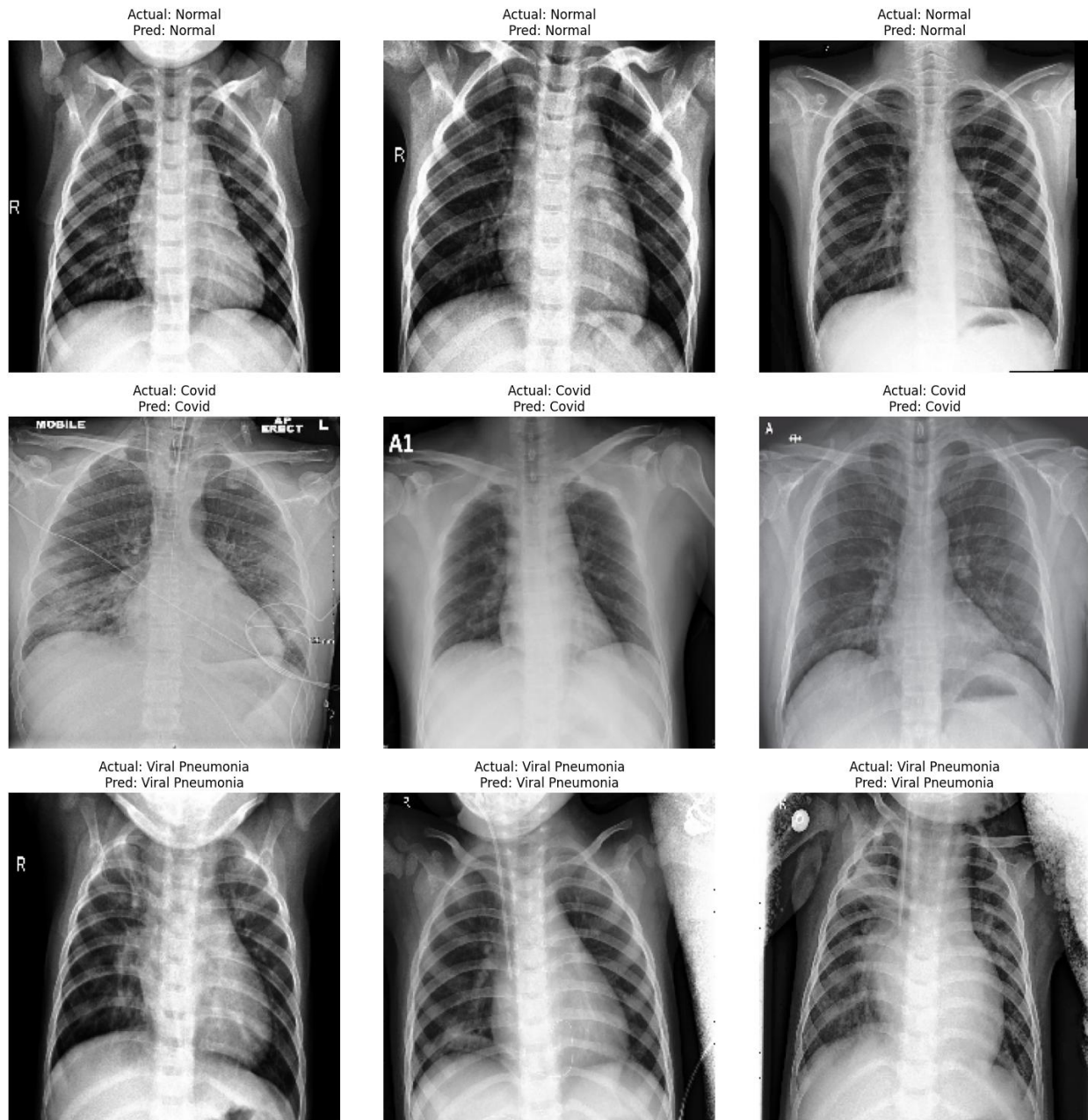
        all_images_to_plot.append(test_img_array)
        all_actual_labels.append(category)
        all_predicted_labels.append(predicted_label)

# Now, plot them in a 3x3 grid
fig, axes = plt.subplots(3, num_samples_per_class, figsize=(15, 15))
axes = axes.flatten()

for i, (img_array, actual_label, predicted_label) in enumerate(zip(all_images_to_plot, all_actual_labels, all_predicted_labels)):
    axes[i].imshow(img_array)
    axes[i].set_title(f"Actual: {actual_label}\nPred: {predicted_label}")
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```

The visual analysis of the model's predictions on chest X-ray images demonstrates its capability to classify cases into 'Normal', 'Covid', and 'Viral Pneumonia'. This qualitative assessment suggests the model's potential as a valuable diagnostic aid.



CONCLUSION

This project successfully developed a Convolutional Neural Network (CNN)-based system for the automatic classification of chest X-ray images into COVID-19, Viral Pneumonia, and Normal categories. Through image preprocessing, data augmentation, and deep learning techniques, the model was able to learn important visual features associated with different respiratory conditions.

The initial CNN model achieved high training accuracy but showed signs of overfitting. To improve generalization, data augmentation, Early Stopping, and Model Checkpointing were incorporated. The modified model achieved a validation accuracy of approximately 96.97%, demonstrating its effectiveness in classifying unseen chest X-ray images.

The results indicate that CNN-based approaches can serve as reliable tools for assisting healthcare professionals in the rapid screening and diagnosis of respiratory diseases. Although the model performed well, its accuracy could be further improved by using larger datasets and more advanced deep learning architectures. Overall, the proposed system demonstrates the potential of artificial intelligence in supporting medical image analysis and disease detection.

FUTURE SCOPE

1. Use larger datasets.
2. Apply transfer learning models such as DenseNet121 and ResNet50.
3. Deploy the model as a web application.
4. Integrate explainable AI techniques.
5. Extend the system to detect multiple lung diseases.

The codes which are applied in this project are stored in this link given below:

<https://github.com/Tanudip572/Chest-XRay-Disease-Classification-CNN>